

基于兼容 Chat Completions API 与本地 VoxCPM 的可控音色语音聊天系统

姓名：_____ 学号：_____

2026 年 6 月 19 日

摘要

本文面向语音信号处理课程大作业，设计并实现了一个基于兼容 Chat Completions API 和本地神经语音合成模型的可控音色语音聊天系统。系统默认使用 DeepSeek API 同时生成朗读文本和本轮语气提示词，也支持替换为其他兼容 `/chat/completions` 端点的模型服务；随后使用本地运行的 OpenBMB VoxCPM/VoxCPM2 将朗读文本转换为语音，并支持自然语言音色设计、参考音频声音克隆和 LoRA 说话人适配。与直接调用在线语音 API 的方案不同，本项目的语音生成模型、数据预处理、微调配置和推理流程均在本地完成，对话 API 仅作为文本和语气规划模块。实验部分计划从可懂度、实时率、音色控制主观评分和不同推理步数下的速度质量权衡进行评估。初步系统分析表明，在 4GB 显存笔记本上完整运行 VoxCPM2 受显存限制，需要采用 CPU 推理、低资源 VoxCPM-0.5B 或更高显存设备。

关键词：语音合成；语音聊天；VoxCPM；DeepSeek；声音克隆；LoRA 微调

1 引言

随着大语言模型和神经语音合成模型的发展，语音交互系统已经从传统的“识别-理解-合成”流水线逐渐转向由大模型驱动的端到端智能交互。课程项目关注深度学习与语音信号处理的结合，本文选择“可控音色语音聊天”作为应用场景：用户输入文本或后续可扩展为语音识别输入，系统由兼容 Chat Completions 的对话模型生成回复内容和语气规划，再通过本地 VoxCPM 合成为带有目标音色的语音。

本项目的目标不是简单调用现成在线语音 API，而是构建一个可复现、可替换、可微调的本地语音生成系统。系统保留 VoxCPM 上游模型作为语音合成后端，在项目侧实现兼容 Chat Completions API 的对话管理、音色控制参数封装、训练数据预处理、LoRA 配置生成和命令行交互入口。这样既能体现开源语音模型复现，也能满足课程对“完整数据预处理、模型定义、训练/推理脚本”的要求。

2 国内外研究现状

早期语音合成常使用拼接合成和统计参数合成，近年来主流方法转向基于深度神经网络的端到端 TTS。Tacotron、FastSpeech、VITS 等模型推动了从文本到梅尔谱再到声码器的神经合成流程。近年的大规模语音生成模型进一步引入离散语音 token、扩散模型、流匹配和语言模型骨干，使系统具备零样本声音克隆、多说话人控制和跨语言生成能力。

VoxCPM 的核心特点是 tokenizer-free，即不依赖外部离散语音分词器，而是在连续语音表征空间中通过层次化语义-声学建模生成语音 [4]。VoxCPM2 在此基础上扩展到 2B 参数、多语言、自然语言音色设计、可控声音克隆和 48kHz 输出 [2]。其官方文档给出了 LocEnc、TSLM、RALM、LocDiT 的四阶段结构，这为课程项目提供了一个具备研究价值的开源语音模型基础。

对话模型方面，默认使用 DeepSeek API 生成语义连贯的文本回复和语气规划 [1]。DeepSeek 在角色扮演和风格化对话上较适合本项目，但工程接口并不绑定 DeepSeek；任意支持兼容 `/chat/completions` 端点的模型服务都可以替换。该模块不属于语音信号处理核心，因此项目将主要实验资源集中在本地 VoxCPM 语音生成、音色控制和 LoRA 适配上。代码中仍保留本地 Hugging Face 对话后端，便于无 API 环境下做替换实验 [3]。

3 方法

3.1 系统总体结构

系统由四个模块组成：兼容 Chat Completions 的对话模块、本地语音合成模型、音色控制层和实验/微调工具链。给定用户输入 x ，对话 API 生成文本回复 y 和语气提示词 p ：

$$(y, p) = f_{\theta}(x, h),$$

其中 h 为历史对话上下文。随后音色控制层将用户固定音色描述 s 、本轮语气提示词 p 与文本回复组合为 VoxCPM2 支持的控制形式：

$$\tilde{y} = \text{concat}((s, p), y).$$

VoxCPM 根据文本、可选参考音频 a_r 和提示音频转写 t_r 生成波形：

$$\hat{a} = g_{\phi}(\tilde{y}, a_r, t_r).$$

图 1 占位：系统流程图。请使用 `paper/figures/prompts.md` 中的 Prompt 1 生成后替换。

图 1: 兼容 Chat Completions API + 本地 VoxCPM 语音聊天系统流程。

3.2 对话 API 模块

项目默认通过 DeepSeek 的 OpenAI 兼容聊天接口生成结构化结果，也可以配置为其他兼容 `/chat/completions` 的模型端点。工程上使用 `tool/function call`，并用 JSON Schema 将工具 `emit_voice_chat_turn` 的参数约束为 `spoken_text`、`voice_prompt` 和 `dialogue_state` 三个字段。第一个字段是真正朗读给用户的内容，第二个字段只描述语气、语速、停顿和情绪，不会被朗读，第三个字段是不进入语音合成的跨轮状态摘要，用于保存角色心境、关系进展、用户偏好和表达基线。系统在每轮调用中保留整段对话历史，包括之前的用户输入、朗读文本、语气提示词和状态摘要，使后续回复同时保持语义连续和表达风格连续。提示词参考 DeepSeek 角色沉浸控制思路，要求模型在私有规划时以角色第一人称把握情绪和表达，但不输出思考链、`<think>` 标签或内心独白；由于 API 通常不会把隐藏思考链传递到下一轮，跨轮连续性由显式的 `dialogue_state` 承担。对话 API 不参与语音合成和音频处理；语音信号处理相关的模型推理、音色控制和 LoRA 适配均在本地 VoxCPM 上完成。代码中保留 `local` 和 `echo` 两种后端，分别用于本地语言模型替换和无模型链路测试。

3.3 VoxCPM 语音合成与音色控制

VoxCPM2 支持两种主要控制方式。第一，自然语言音色设计，即在合成文本前加入括号形式的描述，例如“年轻女性，声音温柔自然，语速中等”。第二，参考音频克隆，即提供参考语音，使模型生成接近参考说话人的音色。项目将用户指定的基础音色或参考音频作为全局设置，在连续对话中保持不变；DeepSeek 每轮只生成语气提示词，用于调整本轮情绪、语速和停顿。项目在 `VoxCPMSynthesizer` 中统一封装模型加载、参数设置、LoRA 权重加载和音频文件输出。

3.4 LoRA 微调

为了适配特定说话人，项目提供 CSV/JSONL 到 VoxCPM manifest 的预处理脚本，并生成上游 LoRA 微调配置。训练样本包含音频路径、转写文本、可选时长和说话人数数据集编号。LoRA 微调只更新低秩适配参数，较适合课程实验中的小规模自录数据。

4 实验设置与结果分析

4.1 实验环境

当前开发机器为 Intel i7-12650H、13GB RAM、NVIDIA RTX 3050 Laptop GPU 4GB。软件环境为 Python 3.11、uv、PyTorch 2.5+、Transformers 和 VoxCPM 上游代码。

4.2 数据与任务

实验计划使用两类数据。第一类为自录普通话语音，时长 5–10 分钟，按句切分并人工转写，用于 LoRA 适配。第二类为开源中文 TTS 数据集中的少量样本，用于对比不同音色控制设置下的合成效果。若使用公开数据集，需在最终提交中补充数据集许可证与下载地址。

4.3 评价指标

实验将记录以下指标：

- 实时率 RTF：合成耗时除以生成语音时长，越低越好。
- 字错率/词错率：使用 ASR 转写合成音频并与目标文本比较。
- 音色相似度：使用说话人嵌入模型计算参考音频与生成音频相似度，或进行主观 MOS 打分。
- 主观自然度：从清晰度、自然度、情感一致性三个维度进行 1–5 分打分。

4.4 初步结果表

由于模型权重和实验音频尚未全部生成，表 1 先给出最终论文需要填写的结果格式。跑通实验后应替换为真实数值。

表 1: 不同设置下的语音生成结果占位表

模型设置	推理设备	RTF ↓	主观自然度 ↑	备注
VoxCPM2 + 音色设计	CPU	待测	待测	质量优先
VoxCPM-0.5B + Prompt 克隆	CPU/GPU	待测	待测	低资源演示
VoxCPM2 + LoRA	CPU/高显存 GPU	待测	待测	个性化音色

5 结论

本文实现了一个基于兼容 Chat Completions API 与本地 VoxCPM 的可控音色语音聊天系统，覆盖了对话生成、语音合成、音色控制、数据预处理和 LoRA 微调配置。系统将在线 API 的作用限制在文本对话与语气规划部分，语音生成模型在本地运行，适合课程复现和后续扩展。硬件分析表明，4GB 显存不足以舒适运行 VoxCPM2，后续实验需要在低资源模型、CPU 推理和高显存设备之间权衡。下一步工作是补充真实音频样例、定量 RTF/ASR 指标、主观评分表，并将生成的科研图替换到论文中。

参考文献

- [1] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [2] VoxCPM Team. Voxcpm2 technical report. *arXiv preprint arXiv:2606.06928*, 2026.
- [3] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- [4] Yixuan Zhou, Guoyang Zeng, Xin Liu, Xiang Li, Renjie Yu, Ziyang Wang, Runchuan Ye, Weiyue Sun, Jiancheng Gui, Kehan Li, Zhiyong Wu, and Zhiyuan Liu. Voxcpm: Tokenizer-free tts for context-aware speech generation and true-to-life voice cloning. *arXiv preprint arXiv:2509.24650*, 2025.

A 核心运行命令

```
uv sync --extra dev
uv pip install -e VoxCPM
export DEEPSEEK_API_KEY="sk-your-key-here"
uv run voiceagent synth --tts-model models/VoxCPM2 --tts-device cpu \
  --voice "young_female_voice, warm_and_natural_tone, medium_pace" \
  --text "你好，这是本地语音聊天系统。" --output outputs/demo.wav
```